DAO

Herencia, composición y polimorfismo

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

CONTENIDOS

Contenido de la clase

01 Herencia

02 Herencia en Python

03 Constructores y métodos en herencia

04 Composición

05 Ejemplos

Herencia

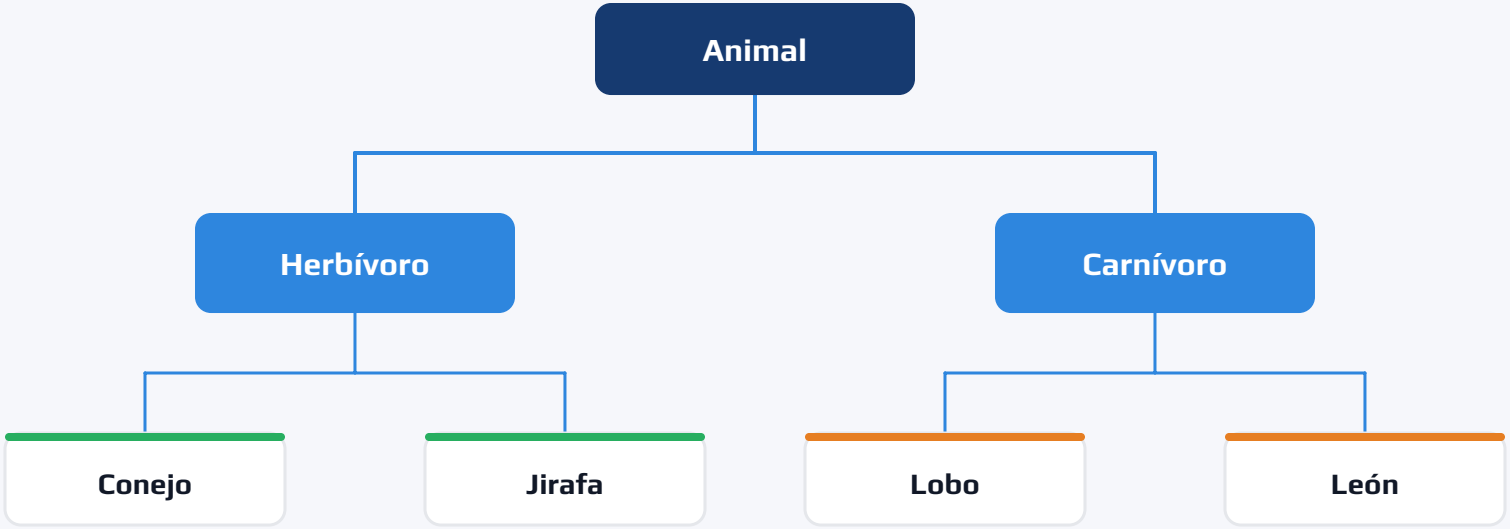
¿Qué es la herencia?

Es el mecanismo por el cual una clase permite heredar las características (atributos y métodos) de otra clase.

Reutilización de Código

La herencia permite que se puedan definir nuevas clases basadas de unas ya existentes a fin de reutilizar el código, generando así una jerarquía de clases dentro de una aplicación.

Herencia: Jerarquía



Herencia: Ejemplo en Python

¿Cómo funciona la herencia?

La subclase (**Coche**) hereda automáticamente los atributos y métodos de la superclase (**Vehiculo**), permitiendo reutilizar y extender el comportamiento.

Puntos clave del ejemplo:

- **Reutilización:** Atributos como `color` y `ruedas` se definen una sola vez en el padre.
- **Especialización:** La clase hija añade atributos propios (`velocidad`, `cilindrada`).
- **Instanciación:** Al crear el objeto de tipo `Coche`, este accede sin problemas a las propiedades heredadas.

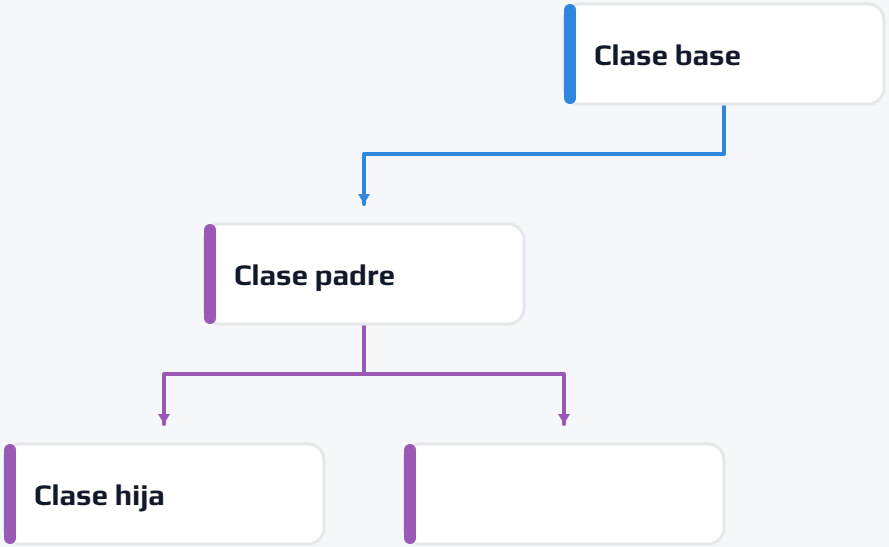
Ejemplo de código

```
class Vehiculo():
    # Cualquier medio de transporte
    color = "Rojo"
    ruedas = 4


class Coche(Vehiculo):
    # Específicamente un vehículo automotor
    velocidad = 200
    cilindrada = 1700


automotor = Coche()
print ("Color:", automotor.color, "Velocidad:",
automotor.velocidad)
```

Terminología



Jerarquía

Relación de orden y dependencia entre las diferentes clases dentro de la estructura de herencia.

Ventajas de la herencia

Principales beneficios de su implementación:

- **Reutilización de código:** Permite aprovechar el comportamiento y atributos ya definidos en las clases padres.
- **Especialización:** Facilita la definición de atributos y comportamientos específicos en las clases hijas.
- **Diseño más simple:** Propicia una arquitectura limpia al asignar las responsabilidades de manera granular.
- **Encapsulamiento más fino:** Permite controlar el acceso a los miembros heredados con mayor precisión.
- **Redefinición flexible:** Otorga la capacidad de sobrescribir comportamientos heredados cuando sea necesario.

Constructores en herencia

¿Cómo se inicializa una jerarquía?

Problema: Si un constructor permite inicializar los valores de una clase particular, surge el desafío de coordinar y propagar esa inicialización a lo largo de toda la jerarquía de clases heredadas.

Desafíos de diseño

- **Atributos heredados:** Definir qué valores deben asumir los atributos que no pertenecen directamente a la clase derivada, sino que se reciben por herencia.
- **Encapsulamiento:** Resolver cómo inicializar el estado de la clase base respetando sus límites, sin manipular sus variables directamente.
- **Mecanismo __init__:** Comprender y estructurar el flujo de ejecución de los constructores en Python al heredar.

Constructores en herencia

Llamada al constructor padre

En Python se puede llamar a los constructores de la clase padre dentro del constructor de la clase derivada utilizando la función `super()`.

Esto permite inicializar correctamente los atributos heredados antes de definir los comportamientos específicos de la subclase.

```python
class Vehiculo():  # Cualquier medio de transporte
  color = None
  ruedas = None
  def __init__(self, color, ruedas):
    self.color = color
    self.ruedas = ruedas

class Coche(Vehiculo):
  # Específicamente un vehículo automotor
  velocidad = None
  cilindrada = None
  def __init__(self, color, ruedas, velocidad, cilindrada):
    super().__init__(color, ruedas)
    self.velocidad = velocidad
    self.cilindrada = cilindrada

```

Métodos en herencia

Relación de Métodos

Los métodos definidos en la clase padre se transmiten a las clases hijas, permitiendo la reutilización y extensión del comportamiento.

- **Uso Directo:** La clase derivada utiliza los métodos de su clase padre como si fueran propios (salvo excepciones).
- **Extensión:** La clase derivada puede definir métodos propios que se agregan a los de la clase padre.

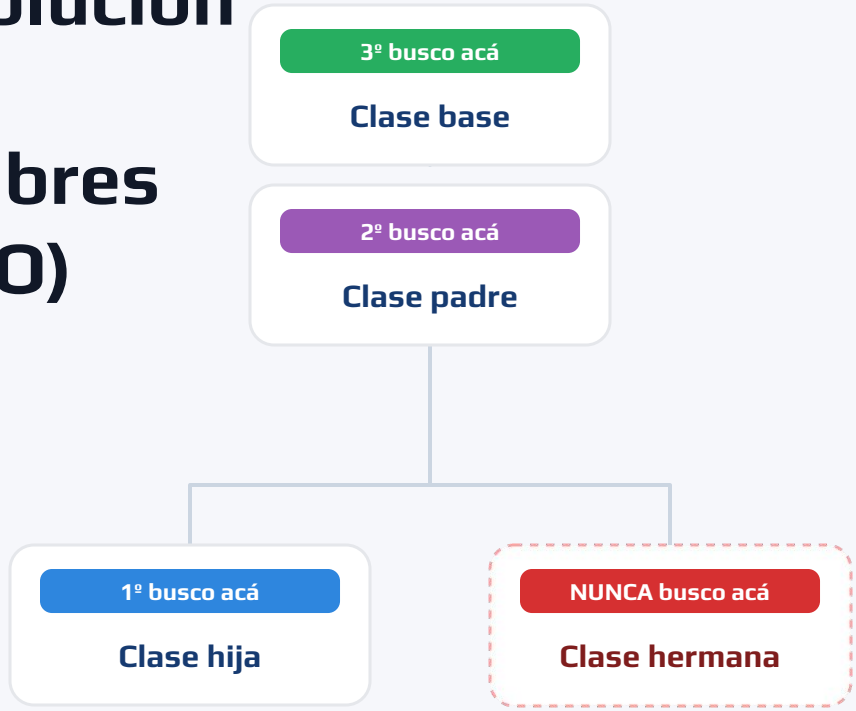
PROBLEMA

¿Conflicto de nombres?

¿Qué pasa si una clase derivada define un método con el **mismo nombre** que un método de la clase padre?

Este escenario introduce el concepto de redefinición o sobrescritura de métodos, fundamental en el polimorfismo.

Resolución de nombres (MRO)



Orden ascendente

La búsqueda de métodos se realiza desde las subclases hacia las superclases.

Métodos en diferentes clases

Herencia y Alcance

Las subclases heredan el comportamiento de la superclase. Sin embargo, los métodos definidos en la subclase no son accesibles por la clase padre.

- **Vehiculo (Padre):** define `avanzar()`. Disponible para ambas clases.
- **Coche (Hijo):** define `cargar_combustible()`. Solo disponible para Coche.

```python
class Vehiculo():
    # Cualquier medio de transporte
    def avanzar (self,distancia):
        pass

class Coche(Vehiculo):
    # Específicamente un vehículo automotor
    def cargar_combustible (self,litros):
        pass

bicicleta = Vehiculo()
automovil = Coche()

bicicleta.avanzar(20) # OK
automovil.avanzar(50) # OK

bicicleta.cargar_combustible(10) # ERROR
automovil.cargar_combustible(10) # OK

```

Métodos Reemplazados (Overriding)

¿Qué es la redefinición?

Es un mecanismo de la POO que permite a una subclase proporcionar una implementación específica de un método que ya está definido en su superclase.

Conceptos clave:

- **Mismo nombre y firma:** El método en la clase hija debe tener exactamente la misma firma que el de la clase padre.
- **Comportamiento dinámico:** Permite que objetos de distintas clases respondan de manera diferente al mismo mensaje.
- **Polimorfismo:** Python decide en tiempo de ejecución qué método invocar según el tipo de objeto.

```python
class Vehiculo():
    # Cualquier medio de transporte
    def get_ruedas (self):
        return 2 # Cantidad mínima de cualquier vehículo

class Coche(Vehiculo):
    # Específicamente un vehículo automotor
    def get_ruedas (self):
        return 4 # Cantidad estándar para cualquier automóvil

bicicleta = Vehiculo()
automovil = Coche()

print (bicicleta.get_ruedas()) # IMPRIME 2
print (automovil.get_ruedas()) # IMPRIME 4
```


UTN FRC UNIVERSIDAD TECNOLÓGICA NACIONAL FACULTAD REGIONAL CÓRDOBA

Métodos encadenados

Extensión de métodos

Permite que una subclase **extienda** la funcionalidad de un método heredado en lugar de reemplazarlo o sobrescribirlo por completo.

Al utilizar **super()**, se invoca el comportamiento original de la clase base para luego sumarle o combinarlo con la lógica particular de la subclase.

```python
class Vehiculo():
    # Cualquier medio de transporte
    autonomia_minima = 10
    def get_autonomia (self):
        return self.autonomia_minima

class Coche(Vehiculo):
    # Específicamente un vehículo automotor
    autonomia = 500
    def get_autonomia (self):
        return super().get_autonomia() + self.autonomia # Autonomía total

bicicleta = Vehiculo()
automovil = Coche()

print (bicicleta.get_autonomia()) # IMPRIME 10
print (automovil.get_autonomia()) # IMPRIME 510
```

Composición: Concepto

Relación

Asociación TIENE-UN

Mientras la herencia representa la relación **ES-UN**, la composición modela la relación **TIENE-UN** entre clases.

Componente

Ciclo de Vida Acoplado

Un componente es parte esencial del objeto contenedor. Su existencia y ciclo de vida están **ligados directamente** al objeto que lo contiene.

Implementación

Objetos Contenidos

En la práctica, la composición se logra instanciando y definiendo referencias a objetos **dentro de otros objetos** de mayor jerarquía.

Composición: Ejemplos

```python
class Motor:
    def __init__(self, autonomia):
        self.autonomia = autonomia

    def get_autonomia(self):
        return self.autonomia


class Coche:
    def __init__(self, capacidad_tanque):
        self.capacidad_tanque = capacidad_tanque
        self.motor = Motor(12)

    def get_autonomia(self):
        return self.motor.get_autonomia() * self.capacidad_tanque


automovil = Coche(50)

print(automovil.get_autonomia())  # IMPRIME 600 (50*12)

```

¿Cómo funciona?

- **Instanciación Interna:** La clase `Coche` crea su propio objeto `Motor` al inicializarse.
- **Relación Fuerte:** El motor depende del ciclo de vida del coche. Si el coche se destruye, el motor también.
- **Delegación:** El método de la clase coche delega el cálculo de la autonomía al método de su motor.

Polimorfismo: La amenaza fantasma

¿Qué es el polimorfismo?

• **Concepto clave:** Es la capacidad que tiene un sistema de clases de que sus tipos respondan a los mismos mensajes, ya sean en su clase base como en sus clases derivadas.

• **Abstracción:** Sirve para abstraer funcionalidad y que el programador no deba preocuparse de qué clase explícita es la que está respondiendo a un mensaje.

• **En Python:** Se aplica de manera natural. Diferentes objetos pueden responder al mismo mensaje siempre que implementen el método correspondiente.



Diseño desacoplado

El cliente envía mensajes sin importar la clase real del receptor.

Polimorfismo - La amenaza fantasma



¿Qué es el polimorfismo?

Se implementa creando métodos en clases derivadas que tienen el **mismo nombre** (no necesariamente los mismos parámetros) que en su clase base.

Regla de oro:

Cualquier clase que herede de **Vehículo** debe responder al mensaje `get_ruedas()`.

Diferentes Implementaciones

A partir de una interfaz o clase base común:

`Vehículo:get_ruedas()`

- Triciclo:`get_ruedas()`
- Bicicleta:`get_ruedas()`
- Automovil:`get_ruedas()`
- Camion:`get_ruedas()`
- *etc.*

DAO

¡Gracias!

Hasta la próxima

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA